

PRICE SIM · System Design Document

Backend Architecture

From raw Hyderabad listings to a serverless, LLM-powered price simulator: data pipeline, model training, artifact exports, TypeScript inference engine, API routes and deployment.

1. System overview

The system is a three-stage pipeline: an **offline Python analysis layer** produces trained models and JSON artifacts, a **Next.js application layer** re-implements those models in TypeScript for instant inference, and a **serverless deployment** (Vercel) serves it with an optional LLM for natural-language explanations.

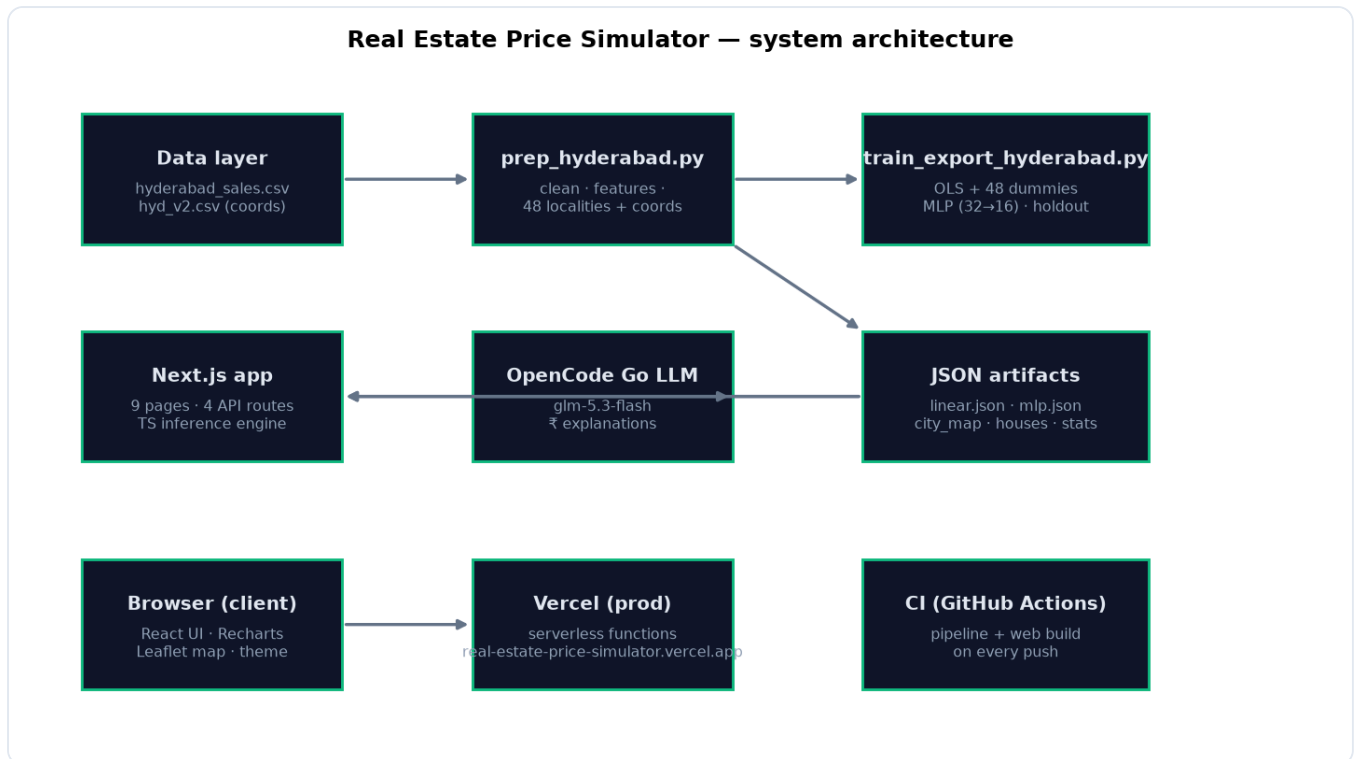


Figure 1 — end-to-end data flow. Everything the app serves is an artifact of the Python pipeline; the web layer only reads artifacts and computes.

2. Data layer

2.1 Sources

File	Rows	Purpose
<code>data/hyderabad_sales.csv</code>	2,518	Primary dataset: price, area, bedrooms, resale, 30 amenity columns, locality
<code>data/hyd_v2.csv</code>	19,110	Rental listings carrying locality coordinates; used once to derive (lat, long) centroids per locality
<code>data/hyderabad_clean.csv</code>	2,518	Output of prep: 11 numeric features + display locality + coordinates

2.2 Feature engineering (`scripts/prep_hyderabad.py`)

- **Binaries:** the 30 amenity columns are counts from the scrape; each is binarised to 0/1 (`value ≥ 1 → 1`).
- **Feature selection:** 11 model features — `area`, `bedrooms`, `resale`, `pool`, `gym`, `clubhouse`, `security`, `backup`, `car_parking`, `lift`, `vaastu` .
- **Locality grouping:** 241 raw locality strings are normalised (lowercase, non-alphanumeric stripped) and grouped into the 48 most frequent named areas plus `other` (49 groups, drop-first coding ⇒ 48 dummies).

- **Coordinates:** each locality maps to the mean (lat, long) of its rental listings; unmatched localities fall back to the city centre (24 rows). A DISPLAY dict restores proper names (“Banjara Hills”, “Hitech City”) after normalisation.
- **Sanity filters:** $300 \leq \text{area} \leq 15,000$ sq ft; $\text{₹}5 \text{ L} \leq \text{price} \leq \text{₹}50 \text{ Cr}$.

3. Model layer

3.1 Training protocol (`scripts/train_export_hyderabad.py`)

Data is split 80/20 (deterministic, `random_state=42`); all metrics below are reported on the 20% holdout. The target is modelled in **INR millions** (`price_scale = 1e6`) for numerical stability; the app multiplies back to rupees at display time.

Model	Method	Holdout R ²	RMSE
Area only	OLS, 1 feature	0.693	₹43.99 L
Multiple (11 features)	OLS	0.709	₹42.86 L
Multiple + locality	OLS + 48 dummy columns	0.775	₹37.68 L
Neural net	MLP 32→16, ReLU, Adam, early stopping	0.714	₹42.46 L

The full model is fitted by **Ordinary Least Squares** on the design matrix $X = [1, \text{features}, \text{dummies}]$:

$$\beta = (X^T X)^{-1} X^T y$$

Locality coefficients are interpretable premia: **Banjara Hills +₹89.9 L**, Jubilee Hills +₹84.1 L, Madhapur +₹41.7 L ... **Attapur -₹24.1 L**, Tellapur -₹21.8 L — relative to the reference locality, controlling for all features.

3.2 Artifacts (bundled into the app at build time)

Artifact	Contents	Consumed by
<code>linear.json</code>	names/coefs/means/stds/intercept, <code>price_scale</code> , <code>zip_prefix="loc_"</code> , lin10 stats (R ² , adj R ² , F, p-values, nobs), holdout R ² /RMSE	predict, breakdown, regression page, coef table, k-NN normalisation
<code>mlp.json</code>	sklearn weights <code>coefs</code> (n_in × n_out) & <code>intercepts</code> , feature means/stds, holdout stats	predict (neural net card), assistant
<code>city_map.json</code>	49 localities: display name, model <code>key</code> , median price, n, lat/long	selects, parser, map chips, similar route
<code>houses.json</code>	2,518 full records (id, price, 11 features, locality, lat/long)	map dots, k-NN similar (bundled import)
<code>stats.json</code> / <code>residuals.json</code>	histograms, locality medians, scatter, correlation matrix, model comparison, residual/QQ samples	explore/correlation/regression pages, <code>/api/stats</code>
<code>figures/*.png</code>	heatmap, residuals, scatter	report page

4. Application layer

4.1 Routes

Route	Method	What it does
/api/predict	POST	Features in → regression + MLP prices, ±1 RMSE band, per-feature ₹ breakdown
/api/similar	POST	k-NN: 10 closest real listings over standardised features
/api/assistant	POST	Text → parser → model → LLM narrative (falls back to template engine)
/api/stats	GET	Aggregated stats, correlation matrix, coefficient table, model comparison

4.2 Inference engine (`web/lib/predict.ts`)

The TypeScript engine reproduces the Python algebra exactly:

- **predictLinear** — mean-centred decomposition: $\text{prediction} = (\text{intercept} + \sum \text{coef} \cdot \text{mean over all columns}) + \sum \text{coef} \cdot (x - \text{mean}) + \text{locality dummy sum}$. The mean-centring constant is rebuilt from the exported arrays so the two sides cannot drift apart.
- **predictMlp** — exact MLP forward pass: feature standardisation, 32→16 ReLU hidden layers, identity output; sklearn matrices are stored ($n_{\text{in}} \times n_{\text{out}}$) and transposed on the fly.
- **nearestHouses** — in-memory k-NN over the standardised 11 features against all 2,518 bundled records ($O(n)$ per query, ~2 ms).

Sample response of `/api/predict` (1,500 sq ft, 3 BHK, amenities, Banjara Hills):

```
{
  "linear": { "price": 16531920, "rmse": 3767898, "r2": 0.7748,
             "band": [12764022, 20299818] },
  "mlp":    { "price": 8244595, "rmse": 4246259, "r2": 0.7140,
             "band": [3998335, 12490854] },
  "breakdown": [ { "key": "location", "label": "Locality", "amount": ... }, ... ]
}
```

4.3 AI assistant flow

1. Regex parser extracts area (sq ft / sq m / sq yd), BHK, resale/new, amenity keywords, and the locality by longest match against `city_map` keys.
2. A feature payload is built and run through the same `predictLinear/predictMlp`.
3. An OpenAI-compatible call is made to **OpenCode Go** (`https://opencode.ai/zen/go/v1/chat/completions`, model `glm-5.3-flash`) with a system prompt that hands the model the computed numbers; the response is the final narrative.
4. If no API key is set (or the call fails), a **template engine** writes the same style of answer from the model output — the app never breaks.

Engineering details that matter: OpenCode Go requires a non-generic `User-Agent` (Cloudflare 403 otherwise) and a stable `x-opencode-session` header; the key lives only in server env vars (never git); the chosen model returns direct content (reasoning models can exhaust the token budget on inner monologue).

4.4 Tax card

All-in cost = price $\times$ (1 + (stamp_duty% + registration%)/100), with rates (4% + 1%) configurable in `web/lib/config.ts` and labelled as approximate Telangana rates.

5. Serverless deployment & operations

5.1 Vercel

- Project `prutxvis-projects/real-estate-price-simulator`, alias **`https://real-estate-price-simulator.vercel.app`**, deployment protection off.
- Every model artifact is **bundled into the serverless functions at build time** — no runtime filesystem reads (Node lambdas cannot read `public/`). This constraint drove the k-NN rewrite to an in-memory import.
- Env vars (API key, base URL, model) are set per environment (production/preview/development).
- Static content (`houses.json`, figures) is served by the Vercel CDN.

```
# deploy
cd web
vercel --prod --yes
# local
npm run build && PORT=3100 npm start
```

5.2 CI (`.github/workflows/ci.yml`)

- **data-pipeline** job: installs the analysis stack, reruns `prep` → `train` → `figures` on every push/PR, so the committed artifacts are always reproducible.
- **web-build** job: `npm ci && npm run build` in `web/`.

5.3 Engineering notes — bugs caught during development

Bug	Symptom	Fix
Raw intercept + mean-centred terms double-counted the mean shift	Every price inflated by ~₹10 L	Rebuild the mean-centring constant from exported arrays
<code>pd.get_dummies(prefix="loc_")</code> adds its own separator	Dummy columns became <code>loc__x</code> ; TS lookup <code>loc_x</code> never matched	Train with <code>prefix="loc"</code> so names align
MLP layers iterated as (out × in) instead of sklearn's (in × out)	NaN prices	Transpose in the forward pass
Map filter defaulted to the dataset minimum (₹20 L)	Map looked empty	Default to ₹2.5 Cr (~94% of listings)
<code>fs.readFileSync(public/...)</code> in a lambda	Would 500 on Vercel	Bundle <code>houses.json</code> via import; in-memory k-NN

6. Repository structure

```
real-estate-price-simulator/  
├─ data/                # raw + cleaned datasets  
├─ scripts/             # prep → train → figures pipeline + requirements.txt  
├─ web/                 # Next.js app (app/, lib/, components/, public/data)  
├─ docs/               # this document, PDFs and figures  
├─ screenshots/        # UI captures for the README  
├─ .github/workflows/ci.yml # pipeline + build CI  
└─ LICENSE · README.md · CONTRIBUTING.md · CHANGELOG.md
```

7. Future work

- Google Maps tiles, transaction (not asking-price) data, and richer amenities such as project age and floor.
- Model families: regularised regression (Ridge/ElasticNet) and tree ensembles for the comparison section.
- Auto-deploy via Vercel git integration and per-commit preview deployments.